La compilation est un ensemble de phases ou de passes qui permettent de partitionner le Programme Source (PS) en ses constituants et d'en créer une représentation intermédiaire (partie analyse), elles permettent aussi de construire le Programme Cible (PC) (partie synthèse).

- La partie analyse comprend : l'analyse lexicale, l'analyse syntaxique et l'analyse sémantique.
  1. **Analyse lexicale** est une analyse linéaire qui, au cours de la lecture du PS caractère par caractère, permet l'élimination des espaces (blancs) et la formation des unités lexicales.

     Exemple : Y := A*X**2 + B.

     Les unités lexicales sont :
     - L'identificateur Y ;
     - Le symbole d'affectation := ;
     - L'identificateur A ;
     - L'opérateur de multiplication * ;
     - L'identificateur X ;
     - L'opérateur de puissance ** ;
     - La constante 2 ;
     - L'opérateur + ;
     - L'identificateur B.

  2. **Analyse syntaxique** est une analyse hiérarchique (parfois grammaticale) qui consiste à regrouper les unités lexicales du PS en structures grammaticales (règles) afin de synthétiser le résultat. Ces structures sont représentées par un arbre syntaxique après la construction d'un arbre abstrait.
     a) Tout identificateur est une expression.
     b) Tout nombre est une expression.
     c) Si $expression_1$ et $expression_2$ et … et $expression_i$ sont des expressions alors : $Expression_1$ op $expression_2$ op … op $expression_i$ est une expression, telle que : op est un opérateur (+, -, *, /, …).

Les règles de base a et b sont non récursives, tandis que la règle c définit des expressions composées d'autres expressions.

Exemple : Y := A*X**2 + B.

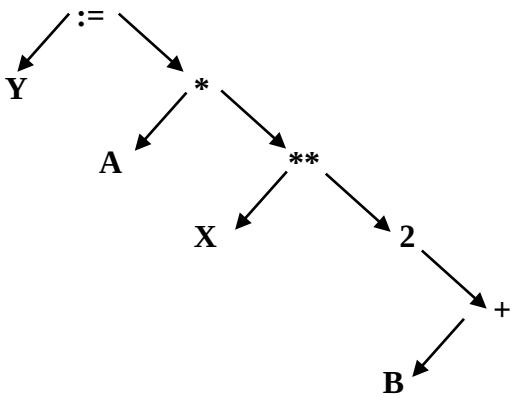


Figure I.1. Arbre abstrait pour Y := A*X**2 + B.

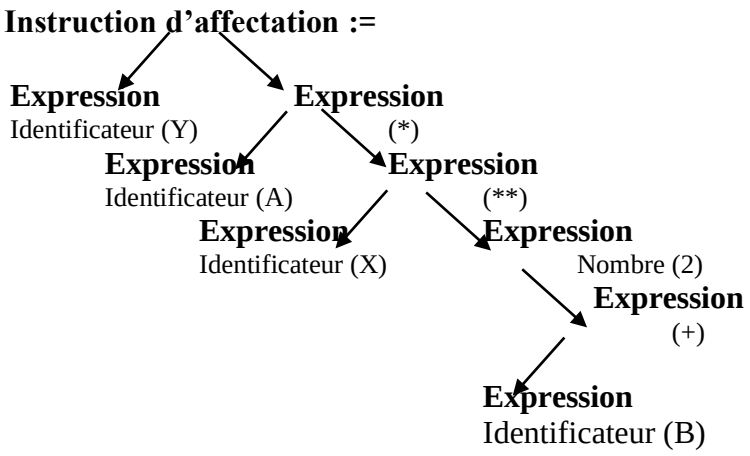


Figure I.2. Arbre syntaxique pour Y := A*X**2 + B.

L'arbre syntaxique est une représentation interne du programme source qui décompacte l'arbre abstrait du même programme source, il est composé des nœuds internes (opérateurs) et des fils du nœud (opérandes).

1. **Analyse sémantique** contrôle si le programme source contient des erreurs sémantiques et collecte des informations de type pour produire le code objet. Elle identifie les opérateurs, les opérandes des expressions et les instructions. Un contrôle important dans cette analyse est la vérification du type, par exemple :
   - Indicer un tableau par un nombre réel,
   - Affecter un réel à un entier,
   - Utiliser une variable non déclarée,
   - … etc.

Exemple d'application :

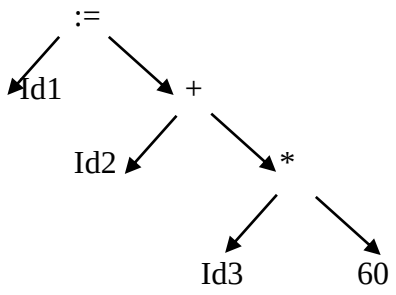
Position := Initiale + Vitesse * 60

1- Analyse lexicale :

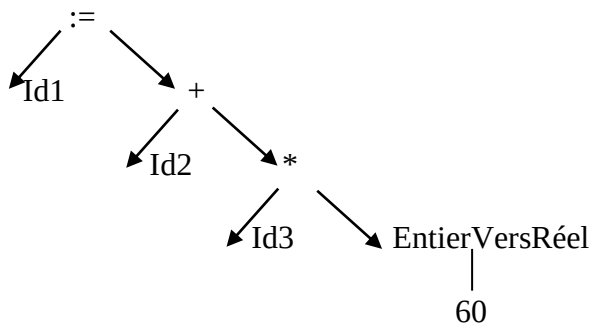
Id1 := Id2 + Id3 * 60

Position est le ***lexème*** de Id1.
Initiale est le ***lexème*** de Id2.
Vitesse est le ***lexème*** de Id3.

2- Analyse syntaxique :



3- Analyse sémantique :



4- Générateur de code intermédiaire :

Temp1 := EntierVersRéel (60)
Temp2 := Id3 * Temp1
Temp3 := Id2 + Temp2
Id1 := Temp3

5- Optimiseur de code :

Temp1 := Id3 * 60.0
Id1 := Id2 + Temp1

6- Générateur de code :

```
MOVF  Id3, R2
MULF  #60.0, R2
MOVF  Id2, R1
ADDF  R2, R1
MOVF R1, Id1
```

Chaque langage de programmation peut être défini par une syntaxe basée souvent sur une grammaire non contextuelle (BNF : Backus-Naur Form) qui peut diriger la traduction des programmes sources, cette technique est appelée traduction dirigée par la syntaxe et par une sémantique en utilisant des descriptions informelles et des exemples.

Dans une compilation à une passe, l'entrée est un flot de caractères et la sortie est une représentation intermédiaire du programme source (voir figure suivante).

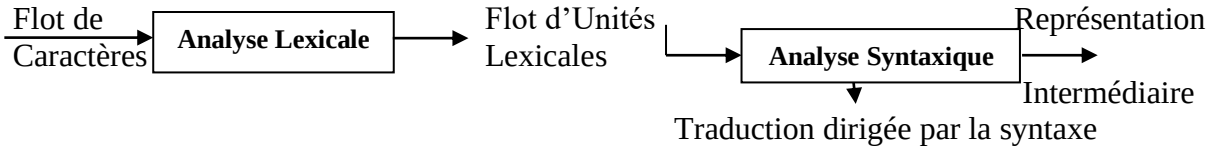


Figure I.3. Compilation à une passe.

- **La syntaxe d'un langage.**

Elle est définie par une grammaire non contextuelle qui est un ensemble de règles de production.

<u>Exemple</u> :

En PASCAL, on écrit : **IF** *conditions* **THEN** *instructions* [**ELSE** *instructions*]

[…] : Pour dénoter l'optionalité de cette partie.

Sa règle est : *instruction* → **IF** *conditions* **THEN** *instructions* [**ELSE** *instructions*]

Où :

→ Signifie : « *peut avoir la forme* ».

Les mots clés : IF, THEN et ELSE sont appelés *unités lexicales* (*Symboles terminaux*).

Les variables : *instruction*, *conditions* et *instructions* sont appelées *non terminaux*.

<u>Notations</u> :

- Les chiffres, les caractères spéciaux (=, >, !, …) et les chaines en gras comme **IF** sont des terminaux.
- Un nom en italique dénote un non terminal.
- Tout nom ou symbole qui n'est pas en italique est supposé être unité lexicale.
- A gauche de la flèche (non terminal) est appelé partie gauche de la règle de production, à droite de la flèche (non terminaux, unités lexicales) est appelé partie droite de la règle.
- L'axiome est le commencement d'une grammaire.
- La chaine de zéro unité lexicale, notée **ε**, est appelée chaine vide.

- **Arbre syntaxique.**

Soit une grammaire non contextuelle, les propriétés d'un arbre syntaxique sont les suivantes :

- L'axiome de la grammaire étiquette la racine.
- Chaque feuille est étiquetée par une unité lexicale ou par **ε**.
- Chaque nœud intérieur est étiqueté par un non terminal.
- Soit A un nœud intérieur avec ses nœuds fils X_1, X_2, …, X_n (de gauche à droite), on aura la règle de production suivante : $A \rightarrow X_1, X_2, \ldots, X_n$ (X_1, X_2, …, X_n peuvent être des terminaux ou des non terminaux).